

BURRACO

Relazione di progetto OOP

A.A 2025/2026

Esposito Beatrice
Ceccaroni Giorgia
Mascioli Sofia
Mazzoli Chiara

INDICE

1. ANALISI	3
1.1 Descrizione e requisiti	3
1.2 Modello del Dominio	6
2. DESIGN	8
2.1 Architettura	8
2.2 Design dettagliato	10
3. SVILUPPO	30
3.1 Testing automatizzato	30
3.2 Note di sviluppo	33
4. COMMENTI FINALI	35
4.1 Autovalutazione e lavori futuri	35
4.2 Sviluppi futuri	37
5. GUIDA UTENTE	38

1. ANALISI

1.1 Descrizione e requisiti

Il software realizzato è una versione digitale a due giocatori, che si sfidano in locale sullo stesso dispositivo, del celebre gioco di carte Burraco.

Il Burraco è un gioco di formazione di combinazioni di carte che si svolge in più round, ovvero è strutturato come una partita articolata in più “sotto-partite”: i giocatori pescano carte, le calano sul tavolo in sequenze o tris, e cercano di chiudere il round svuotando la mano (dopo aver soddisfatto determinate condizioni).

Configurazione iniziale e gestione della partita

All'avvio della partita, i giocatori possono configurare le impostazioni iniziali tramite un apposito menu, definendo i propri nomi e la soglia di punteggio necessaria per decretare la vittoria finale. Il sistema gestisce l'intero ciclo di vita della partita: dalla preparazione del mazzo comune, alla distribuzione iniziale delle carte ai giocatori, alla creazione del monte scarti, e alla pesca di nuove carte.

Meccaniche di turno

Durante il proprio turno, il giocatore ha la possibilità di effettuare due operazioni principali per pescare nuove carte: prendere la prima carta dal mazzo coperto oppure raccogliere l'intero ammontare delle carte presenti nel monte scarti. L'obiettivo principale è creare combinazioni valide (sequenze dello stesso seme o gruppi dello stesso valore) da calare sul tavolo o "attaccare" carte a combinazioni già esistenti per aumentare il proprio punteggio. Il gioco include la gestione di carte speciali come i Jolly e le Pinelle (2), che possono sostituire qualsiasi altra carta nelle combinazioni.

Gestione dei pozzetti

Il cuore della strategia risiede nella gestione dei pozzetti, due mazzi da undici carte preparati all'inizio della smazzata. Il primo obiettivo di ogni giocatore è esaurire le carte ricevute inizialmente per poter accedere a questo mazzo supplementare. La transizione può avvenire tramite "presa diretta", detto “pozzetto al volo” se il giocatore cala l'ultima carta continuando a giocare immediatamente, o "con scarto", se l'esaurimento della mano avviene tramite lo scarto di una carta nella pila degli scarti, rimandando l'utilizzo del pozzetto al turno successivo. Questa fase è determinante anche per evitare la penalità di 100 punti prevista per chi non raggiunge il pozzetto, una volta che uno dei due giocatori termina definitivamente la partita “chiudendo”.

L'obiettivo principale è totalizzare il punteggio stabilito per la vittoria finale prima degli avversari, attraverso la creazione di combinazioni (carte dello stesso valore) o sequenze (carte dello stesso seme in ordine numerico).

Condizioni di chiusura

La chiusura definitiva richiede invece il soddisfacimento di tre fattori contemporanei: il giocatore deve aver già preso il proprio pozzetto ed averlo "esaurito", il giocatore deve aver realizzato almeno un Burraco (una combinazione di almeno sette carte) e il giocatore di turno deve restare senza carte scartando la carta finale nel monte degli scarti.

Calcolo del punteggio e fine partita

Una volta avvenuta la chiusura, il sistema procede all'assegnazione dei punteggi, sommando il valore delle carte a terra, cioè le combinazioni presenti sul tavolo di gioco, e i bonus, sottraendo i punti delle carte rimaste in mano agli avversari. La sfida prosegue con altre partite, finché una delle due parti non raggiunge o supera la soglia di punteggio stabilita inizialmente. Solo in quel momento il software interrompe il ciclo di gioco e proclama il vincitore finale in base al punteggio complessivo maturato.

Glossario dei termini di dominio

Mano: L'insieme di carte attualmente possedute da un giocatore, visibili solo a lui.

Mazzo: Il mazzo comune da cui si pesca durante il gioco. Comprende due mazzi da 52 carte più 4 jolly (108 carte totali).

Pila degli scarti: Le carte scartate dai giocatori, visibili a entrambi; il giocatore in turno può prenderle tutte in un colpo solo.

Pozzetto: Un mazzo di 11 carte coperte assegnato a ciascun giocatore all'inizio della partita. Si può prendere solo dopo aver esaurito le carte in mano.

Combinazione: Un insieme di carte calate sul tavolo, può essere:

- una scala (carte dello stesso seme in sequenza; es Asso, Re, Donna di cuori)
- un set (carte dello stesso valore; es Nove di cuori, Nove di fiori, Nove di picche).

Burraco: Una combinazione di almeno 7 carte. È pulito se non contiene jolly né 2 usati come sostituti di carte, sporco altrimenti. Completare almeno un burraco è necessario per poter chiudere il round.

Chiusura: Azione con cui un giocatore termina il round scartando l'ultima carta, dopo aver preso il pozzetto e completato almeno un burraco.

Round: Una singola manche della partita. Al termine, si calcolano i punti; la partita prosegue con un nuovo round finché un giocatore raggiunge o supera il punteggio obiettivo.

Punteggio obiettivo: Soglia di punteggio cumulativa (scelta a discrezione dell'utente) che, se raggiunta da almeno un giocatore, termina l'intera partita.

Requisiti funzionali

- **Gestione dello Start della partita:** All'avvio dell'applicazione deve essere possibile configurare i nomi dei giocatori e il punteggio obiettivo.
- **Gestione della partita:** Il sistema deve permettere a due giocatori di giocare una partita completa di Burraco, composta da più round consecutivi.
- **Gestione del Mazzo e Distribuzione:** Il sistema deve gestire due mazzi di carte, occuparsi del mescolamento e della distribuzione di 11 carte a ogni giocatore per ogni round.
- **Gestione del turno e la loro alternanza:** Ciascun giocatore deve poter compiere diverse azioni durante il suo turno: pescare (dal mazzo o dalla pila degli scarti), calare combinazioni, “attaccare” carte a combinazioni già presenti, scartare una carta per terminare il turno.
- **Logica di validazione:** verificare automaticamente la validità delle combinazioni create e la possibilità di aggiungere carte a gruppi già presenti sul tavolo di gioco, impedendo mosse illegali e notificando il giocatore in caso di errore.
- **Gestione dei Pozzetti:** Creazione di due mazzetti ("pozzetti") di 11 carte ciascuno ad inizio partita, da assegnare automaticamente ad ogni giocatore una volta che esaurisce le carte che ha nella sua mano di partenza.
- **Gestione della chiusura del round:** Il sistema deve verificare le condizioni di chiusura del round (pozzetto preso, almeno un burraco) e terminare il round solo quando queste sono soddisfatte.
- **Sistema di punteggio:** Il sistema deve calcolare il punteggio al termine di ogni round (bonus burraco pulito/sporco, bonus chiusura, penalità per mancato pozzetto, detrazione carte rimaste in mano). Il sistema deve inoltre mantenere un punteggio cumulativo per tutta la durata della partita e dichiarare il vincitore quando un giocatore raggiunge o supera il punteggio obiettivo

Requisiti non funzionali

- **Interfaccia utente reattiva e intuitiva:** l'interfaccia grafica deve rispondere alle azioni dell'utente in modo fluido e senza blocchi. Il sistema fornisce inoltre una rappresentazione grafica chiara del tavolo da gioco, della mano del giocatore e delle aree comuni (mazzo e scarti) per facilitare l'esperienza utente.
- **Portabilità:** Il software deve essere eseguibile su diverse piattaforme desktop che supportino l'ambiente di esecuzione scelto.
- **Robustezza:** il sistema è progettato per gestire in modo sicuro input errati e combinazioni non valide, impedendo qualsiasi blocco o interruzione del flusso di gioco. Ogni azione non consentita viene rilevata e segnalata all'utente tramite un opportuno messaggio, consentendo alla partita di proseguire correttamente senza compromettere lo stato del gioco.

1.2 Modello del Dominio

Il sistema dovrà gestire l'entità **Game**, articolata in più **Round** consecutivi, che coinvolge due **Player** identificati da un nome scelto in fase di configurazione. La Partita è caratterizzata da un **Target Score**, la cui soglia, una volta raggiunta, decreta la vittoria finale.

Ogni Round si articola in una sequenza di **Turn** alternato tra i due giocatori e si svolge attorno a un **Table** condiviso, che aggrega il **Deck** (108 carte: due mazzi da 52 più 4 Jolly), la **Discard Pile** e le **Combination** calate dai giocatori. Ciascun Giocatore gestisce una **Hand**, ovvero l'insieme delle carte in suo possesso, e un **Pot**, un mazzo coperto di undici carte accessibile solo dopo aver esaurito la mano iniziale.

L'elemento fondamentale è la **Card**, caratterizzata da un seme e un valore. Rivestono un ruolo speciale i Jolly e le Pinelle (i 2 di qualsiasi seme), che possono sostituire qualsiasi carta in una Combinazione. Le Combinazioni si distinguono in **Straight** (stesso seme in sequenza) e **Set** (stesso valore, seme diverso); una combinazione di almeno sette carte costituisce un Burraco.

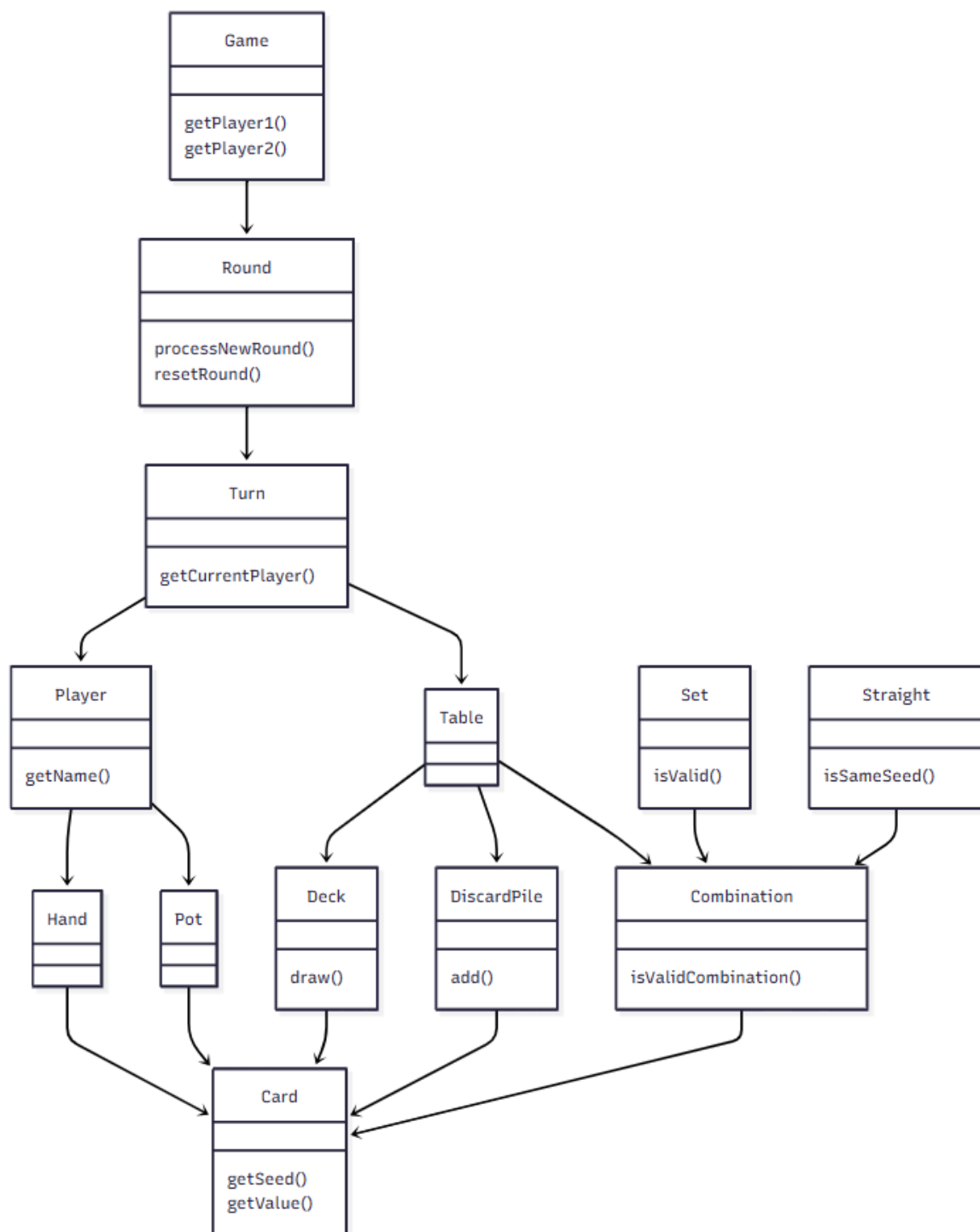


Figura 1.1: Schema UML del modello del dominio

2. DESIGN

2.1 Architettura

Il progetto è stato sviluppato seguendo il pattern architetturale **Model-View-Controller (MVC)**, una scelta che si è rivelata particolarmente adatta alla natura a turni del gioco, in cui lo stato del sistema evolve in modo sequenziale e ben definito. Il cuore del sistema è un **Game Loop** centrale, gestito dal livello Controller, responsabile di sincronizzare la logica applicativa (Model) con la rappresentazione grafica (View).

I tre livelli dell'architettura sono definiti dalle seguenti interfacce principali:

1. **Model (GameModel)**: Costituisce il nucleo computazionale dell'applicazione. Al suo interno risiedono la gestione della partita, l'alternanza dei round, le condizioni di chiusura e il calcolo del punteggio cumulativo. Il Model non ha alcuna dipendenza nei confronti della View o del Controller, garantendo così la massima separazione delle responsabilità.
2. **View (BurracoView)**: Si occupa esclusivamente della visualizzazione e della raccolta degli input. Non contiene alcuna logica decisionale: il suo compito è visualizzare le informazioni ricevute e trasmettere al Controller le interazioni dell'utente, come la selezione di una carta, la pesca dal mazzo o il calo di una combinazione sul tavolo.
3. **Controller (GameController)**: Svolge il ruolo di coordinatore dell'intera applicazione. Il ciclo di vita della partita è gestito dal [GameLoopController](#), mentre responsabilità più specifiche sono delegate a componenti dedicati.

Interazione e Flusso Dati

Dal Model alla View: Il Model non interagisce mai direttamente con la View. Dopo ogni azione dell'utente o iterazione del loop, è il Controller a prelevare lo stato aggiornato dal [GameModel](#) e a trasmetterlo alla [Burraco View](#). A tal fine è stato introdotto il GameState, un oggetto DTO immutabile che funge da istantanea dello stato di gioco. Questo approccio impedisce alla View di accedere direttamente al Model, eliminando il rischio di modifiche accidentali alla logica applicativa.

Dalla View al Controller: Le interazioni dell'utente vengono intercettate esclusivamente dalla [BurracoView](#). Internamente, la view utilizza il [SelectionCardManager](#) per tenere traccia delle carte selezionate dal giocatore nella propria mano. Quando l'utente conferma un'azione, la view costruisce un oggetto [ViewAction](#) e completa un `CompletableFuture<ViewAction>` iniettato dal controller. Il [GameLoopController](#), in attesa su quel future nel proprio thread, riceve il ViewAction e lo traduce in un oggetto Move del model.

Gestione dei Turni: Il [GameLoopController](#) si occupa di alternare i turni tra i due giocatori. A ogni turno, il Controller attende gli input provenienti dalla View prima di procedere. Le condizioni di chiusura del round sono verificate internamente dal Model, tramite il [ClosureValidator](#) invocato all'interno di [GameModelImpl](#) ad ogni applicazione di una mossa. Quando il Model segnala la fine del round attraverso il [MoveResult](#), il [GameLoopController](#) delega allo [ScoreController](#) il calcolo e l'aggiornamento dei punteggi, e al [RoundControllerImpl](#) il ripristino dello stato per la manche successiva tramite [ResetManager](#). In caso di mossa non valida, il Controller traduce il [MoveResult.Status](#) restituito dal Model in un [MoveError](#), un enum del layer view, prima di notificare la [BurracoView](#). Questo garantisce che la View non abbia mai dipendenze dirette dai tipi del Model, rendendo i due layer completamente indipendenti.

Uno dei vantaggi principali di questa struttura è la sua flessibilità: sostituire completamente l'interfaccia grafica richiederebbe di intervenire unicamente sull'implementazione di [BurracoView](#) e [SoundView](#), senza toccare né il [GameModel](#) né la logica di controllo. Questo è reso possibile dal fatto che il Controller dialoga con la View esclusivamente attraverso le interfacce [BurracoView](#) e [SoundView](#), restando del tutto indipendente dalla tecnologia grafica impiegata.

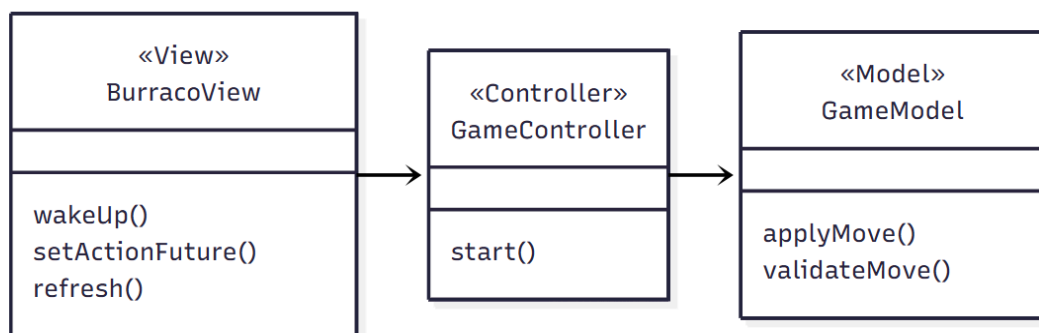


Figura 2: Rappresentazione UML architetturale di Burraco

2.2 Design dettagliato

Beatrice Esposto

Inizializzazione e consistenza del mazzo

Descrizione del problema

Il gioco del Burraco richiede l'unione di due mazzi standard da 52 carte più 4 Jolly, per un totale di 108 carte. La sfida tecnica non risiede nella semplice concatenazione dei due mazzi, ma nella necessità di garantire che ogni carta sia trattata come un'istanza univoca e distinguibile durante l'intera vita del round. Senza un meccanismo di tracciamento dell'unicità, operazioni come la distribuzione delle carte ai giocatori e la creazione dei pozzetti avrebbero potuto produrre duplicati, causando inconsistenze di stato difficili da diagnosticare.

Soluzione proposta

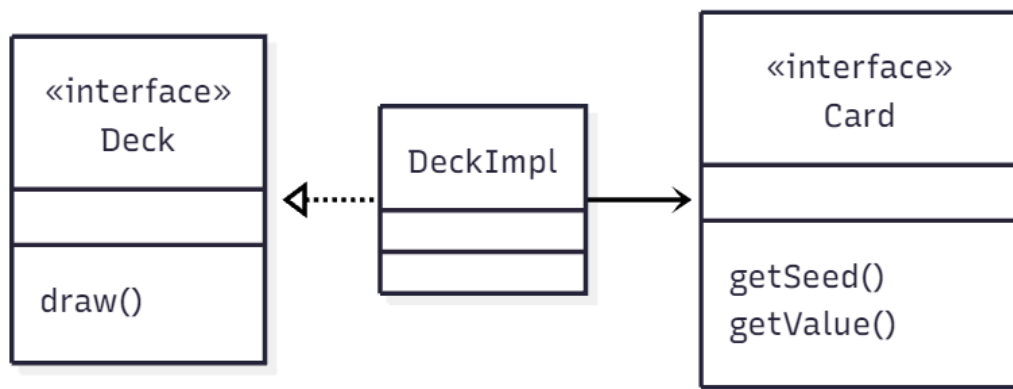
La classe [DeckImpl](#) si occupa della costruzione del mazzo combinato, istanziando ogni carta come oggetto distinto anche quando due carte condividono seme e valore (appartenendo ai due mazzi diversi). La mescolazione avviene tramite un algoritmo di shuffle sull'intera collezione, dopo la quale le operazioni di distribuzione avvengono sempre per rimozione dalla cima del mazzo, garantendo che ogni carta venga assegnata una sola volta. L'intero ciclo di vita del mazzo è incapsulato nella classe, che non espone mai la collezione interna direttamente.

Analisi della soluzione proposta

Incapsulare la logica di costruzione e distribuzione all'interno di [DeckImpl](#) ha permesso di isolare completamente il resto del sistema dalla complessità della gestione del mazzo. Qualsiasi componente che necessiti di una carta la ottiene tramite il metodo `draw()`, senza poter accedere o alterare la struttura interna della collezione.

Analisi delle alternative scartate

Si era valutata l'ipotesi di costruire il mazzo direttamente nel [GameModel](#) tramite una lista inizializzata manualmente. Questa strada è stata abbandonata perché mescolava responsabilità di configurazione con responsabilità di gestione del flusso di gioco, rendendo il codice del Model più difficile da leggere e testare.



Gestione dell'ambiguità semantica della carta "2" (Pinella)

Descrizione del problema

Nelle scale del Burraco, la carta "2" presenta una duplice natura: può occupare la sua posizione naturale nella sequenza (tra l'Asso e il 3) oppure comportarsi come una matta, sostituendo qualsiasi carta mancante all'interno della scala. Il problema principale consiste nel determinare programmaticamente quale dei due ruoli debba assumere in un dato contesto, poiché la validità dell'intera scala dipende proprio da questa distinzione.

Soluzione proposta

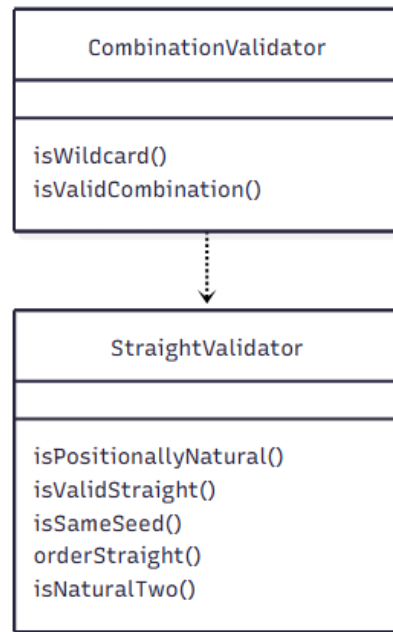
All'interno di [StraightValidator](#) è stata implementata una logica di risoluzione dell'ambiguità che analizza il contesto della scala prima di assegnare un ruolo al "2". La validazione viene eseguita in due passi: nel primo il '2' viene trattato come carta naturale se compatibile con la sequenza; nel secondo viene forzato come matta. La combinazione è valida se almeno una delle due opzioni ha esito positivo. Questo approccio garantisce che la carta venga promossa a jolly esclusivamente quando necessario, riducendo i falsi negativi nella validazione.

Analisi della soluzione proposta

Centralizzare questa logica in [StraightValidator](#) ha evitato di duplicare il ragionamento sull'ambiguità del "2" in [CombinationValidator](#), che si limita a delegare la risoluzione al metodo condiviso. Questo ha reso il codice più coerente e ha semplificato notevolmente i test, che possono verificare il comportamento del "2" in isolamento senza dover simulare un'intera combinazione.

Analisi delle alternative scartate

Si era considerata l'opzione di trattare il "2" sempre come matta a livello di modello, eliminando l'ambiguità alla radice. Questa scelta è stata scartata perché avrebbe penalizzato i casi in cui il "2" occupa la sua posizione naturale.



Validazione degli attacchi a combinazioni esistenti

Descrizione del problema

Quando un giocatore vuole aggiungere carte a una combinazione già presente sul tavolo, il sistema deve verificare che la combinazione risultante rimanga valida secondo le regole del Burraco. Questo include i casi in cui le nuove carte si aggiungono a scale che contengono già matte, richiedendo che la combinazione risultante rispetti i limiti e mantenga la sequenza corretta. La sfida tecnica consiste nel validare l'operazione senza duplicare la logica già esistente per le nuove combinazioni.

Soluzione proposta

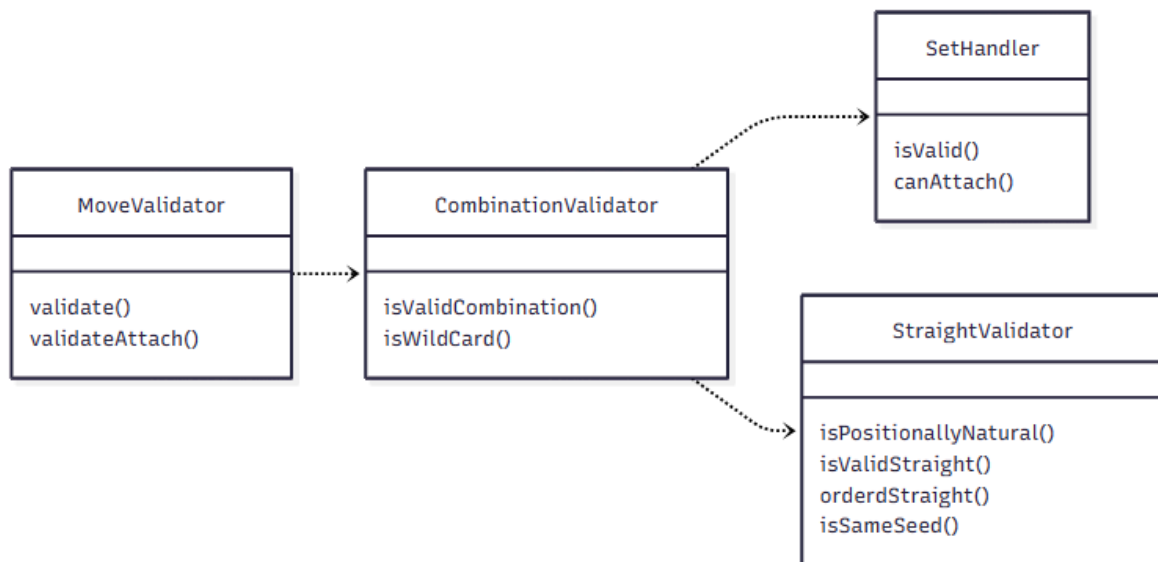
Il contributo specifico in questa sezione riguarda la logica di validazione delle scale implementata in [StraightValidator](#) e [CombinationValidator](#). Quando un giocatore vuole aggiungere carte a una combinazione esistente, [MoveValidator](#) costruisce ipoteticamente la combinazione risultante aggiungendo le carte selezionate a quelle già presenti sul tavolo, e delega la verifica a `isValidCombination()`. Questa delega funziona correttamente perché [CombinationValidator](#) applica le stesse regole sulle scale sia per le nuove combinazioni che per quelle ipotetiche risultanti da un attacco.

Analisi della soluzione proposta

Questo approccio garantisce coerenza tra la fase di creazione di una combinazione e quella di attacco a una esistente: qualsiasi modifica alle regole di validazione delle scale si propaga automaticamente a entrambi i contesti senza dover intervenire in punti diversi del codice.

Analisi delle alternative scartate

Si era valutata l'ipotesi di memorizzare esplicitamente nella matra il valore che sta sostituendo al momento del calo. Questa soluzione è stata scartata perché introduceva uno stato mutabile nell'oggetto Card, complicando la gestione delle copie e aumentando il rischio di inconsistenze quando la stessa carta veniva referenziata in contesti diversi.



Sofia Mascioli

Gestione dinamica del layout del tavolo da gioco

Descrizione del problema

Nel corso di un round di Burraco il numero di combinazioni presenti sul tavolo cresce dinamicamente. La sfida consiste nel mantenere la visualizzazione sempre sincronizzata con lo stato del Model, mantenendo distinta l'area di gioco dei due giocatori e garantendo che l'ordinamento delle combinazioni sia coerente indipendentemente dall'ordine in cui sono state calate.

Soluzione proposta

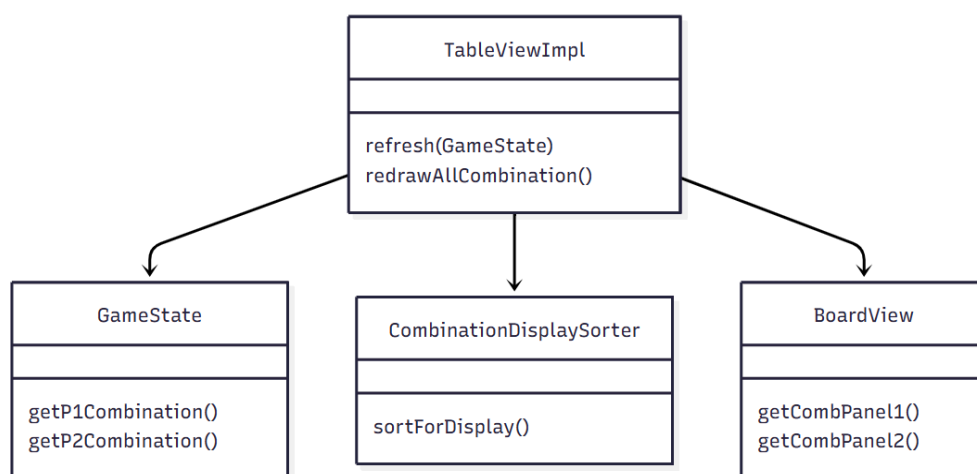
La visualizzazione del tavolo è gestita da [TableViewImpl](#), che ad ogni aggiornamento riceve il [GameState](#) dal Controller e invoca redrawAllCombinations, che rimuove tutti i componenti presenti nei pannelli dei due giocatori e li ridisegna interamente. Prima del rendering, [CombinationDisplaySorter](#) ordina le combinazioni garantendo una presentazione coerente. La separazione tra logica di ordinamento e logica di rendering permette di modificare il criterio di visualizzazione senza toccare il componente grafico.

Analisi della soluzione proposta

Ridisegnare il pannello a partire dal [GameState](#) aggiornato garantisce che la View sia sempre perfettamente sincronizzata con il Model, senza rischi di disallineamento tra lo stato interno e quello visualizzato.

Analisi delle alternative scartate

Si era valutata l'ipotesi di aggiornare incrementalmente solo le combinazioni modificate, confrontando lo stato precedente con quello nuovo. Questa soluzione è stata scartata perché richiedeva di mantenere uno stato interno nella View, aumentando la complessità e il rischio di disallineamento.



Gestione della logica di validazione Set vs Scale

Descrizione del problema

La sfida principale nella validazione delle combinazioni risiede nella necessità di gestire due tipologie di regole strutturalmente eterogenee: i set o "tris", composti da carte dello stesso valore ma di seme diverso, e le scale, composte da carte dello stesso seme in sequenza numerica crescente. Le due tipologie condividono l'interfaccia di validazione ma seguono criteri completamente diversi, e la complessità è ulteriormente accentuata dal ruolo duale del "2", che può agire come carta naturale nelle scale ma anche come matta in entrambe le tipologie.

Soluzione proposta

La logica di validazione è stata separata in base alla tipologia di combinazione. [CombinationValidator](#) funge da punto di ingresso unificato e, ricevuta una combinazione, ne analizza le caratteristiche strutturali per indirizzarla al percorso di validazione corretto: se rileva valori duplicati tra le carte naturali, la tratta come set

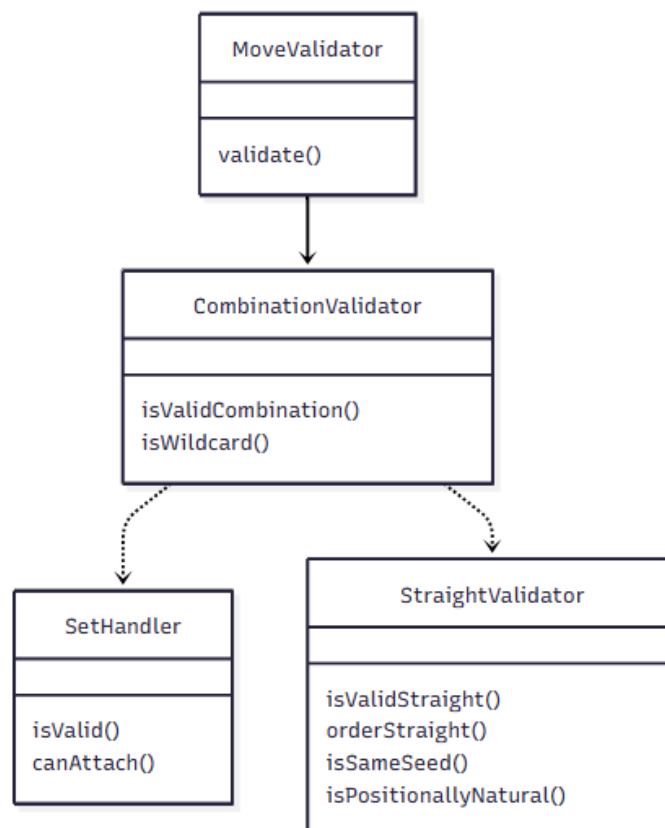
delegando a [SetHandler](#); se le carte condividono lo stesso seme, la tratta come scala delegando a [StraightValidator](#).

Analisi della soluzione proposta

La separazione tra la validazione dei set e quella delle scale ha reso il codice più leggibile e ha permesso di testare le due logiche in modo completamente indipendente. Aggiungere in futuro una nuova tipologia di combinazione richiederebbe di estendere [CombinationValidator](#) con un nuovo percorso di delega, senza modificare la logica già esistente e verificata, rispettando il principio Open-Closed.

Analisi delle alternative scartate

Si era valutata l'idea di implementare un unico metodo di validazione con rami condizionali distinti per gruppi e scale. Questa soluzione è stata scartata perché all'aumentare dei casi speciali il metodo sarebbe diventato rapidamente ingestibile, con condizioni annidate difficili da seguire e un elevato rischio di regressioni durante le modifiche. La separazione delle responsabilità ha invece permesso di affrontare ciascun caso limite nel contesto appropriato, con piena visibilità sul sottoinsieme di regole coinvolto.



Gestione della dualità posizionale dell'Asso nelle scale

Descrizione del problema

Nelle scale del Burraco, l'Asso costituisce un'eccezione alla normale gerarchia numerica: può assumere due posizioni distinte all'interno di una sequenza, fungendo da "Asso basso" (valore 1, che precede il 2 naturale) oppure da "Asso alto" (valore 14, successivo al Re). Questa dualità introduce una complessità nella validazione dell'operazione di attacco: quando un giocatore tenta di aggiungere un Asso a una scala già presente sul tavolo, il sistema deve determinare dinamicamente quale dei due ruoli l'Asso stia assumendo in quel contesto, verificando che completi correttamente una delle due estremità della sequenza senza romperne la coerenza.

Soluzione proposta

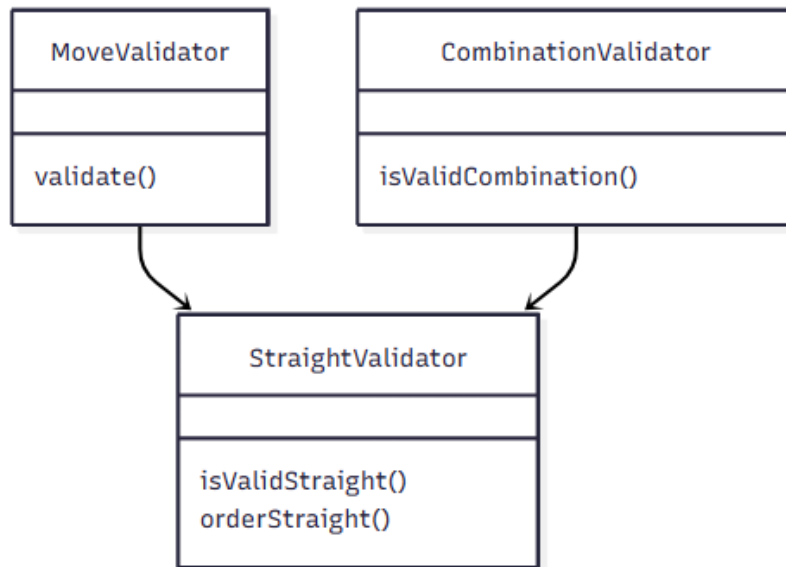
La gestione della dualità dell'Asso è stata centralizzata in [StraightValidator](#). Ogni volta che viene validata una scala, sia in fase di creazione che di attacco, il sistema esegue la verifica due volte: una trattando l'Asso come valore 1 e una come valore 14. La combinazione è considerata valida se almeno una delle due interpretazioni produce una sequenza corretta. Poiché sia [CombinationValidator](#) che [MoveValidator](#) delegano la validazione a [StraightValidator](#), il comportamento è garantito coerente in entrambi i contesti.

Analisi della soluzione proposta

Risolvere il valore dell'Asso in modo contestuale ha permesso di gestire correttamente tutti i casi senza introdurre stati mutabili nell'oggetto Card. La centralizzazione in [StraightValidator](#) ha inoltre garantito coerenza tra la fase di validazione di una nuova combinazione e quella di attacco a una combinazione esistente, evitando che lo stesso Asso venisse trattato diversamente nei due contesti.

Analisi delle alternative scartate

Si era valutata l'ipotesi di introdurre nella classe Card un attributo esplicito per il valore posizionale dell'Asso, da impostare al momento del calo. Questa soluzione è stata scartata perché avrebbe reso la carta stateful, complicando la gestione delle istanze e introducendo il rischio che il valore posizionale impostato in un contesto interferisse con la validazione in un contesto diverso. Mantenere la risoluzione del valore all'esterno della carta, come responsabilità del validatore, ha prodotto un design più pulito e privo di effetti collaterali.



Chiara Mazzoli

Mapping del valore delle carte e calcolo dei punti

Descrizione del problema

Nel Burraco il valore di una carta ai fini del punteggio finale non corrisponde al suo valore numerico: un Asso vale 15 punti, un Jolly 30, una Pinella 20, le figure 10 ciascuna e le carte numeriche hanno valori variabili. Inserire questa logica direttamente nella classe `Card` avrebbe violato il principio di singola responsabilità, appesantendo il modello con una preoccupazione strettamente legata alla fase di calcolo finale e rendendo la classe più difficile da riutilizzare in altri contesti.

Soluzione proposta

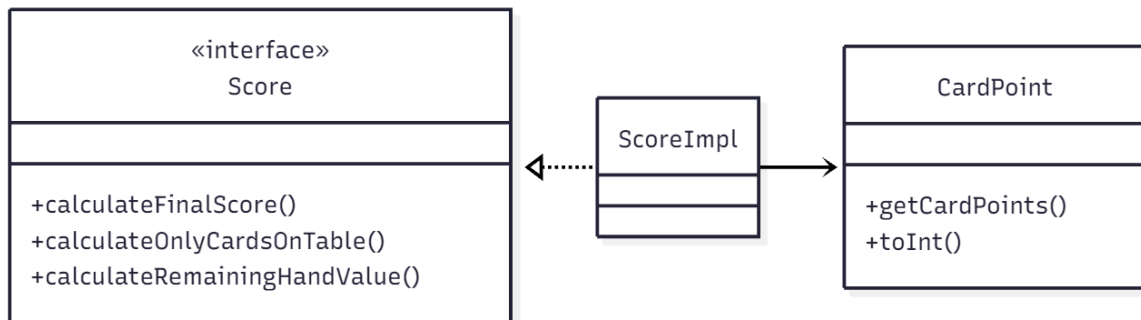
La conversione tra carta e punteggio è stata centralizzata nella Utility Class `CardPoint`, che centralizza la logica di conversione. Questa classe espone un metodo che riceve una `Card` e restituisce il valore in punti corrispondente, incapsulando completamente la tabella di conversione. `ScoreImpl` delega a `CardPoint` ogni operazione di valorizzazione, senza conoscere i dettagli della tabella di mapping. `ScoreController` a sua volta conosce solo l'interfaccia `Score`, restando completamente disaccoppiato dalla logica di conversione.

Analisi della soluzione proposta

Separare la logica di valorizzazione dalla classe `Card` ha mantenuto il modello snello e focalizzato sulla rappresentazione della carta come entità di gioco. Modificare i punteggi richiede di intervenire esclusivamente su `CardPoint`, senza toccare né `Card` né `ScoreController`.

Analisi delle alternative scartate

Si era valutata l'ipotesi di inserire un metodo `getPoints()` direttamente nella classe `Card`. Questa soluzione è stata scartata perché avrebbe introdotto nella classe una dipendenza dalle regole di punteggio, che sono una preoccupazione del dominio del calcolo finale e non della rappresentazione della carta durante il gioco.



Identificazione e classificazione del Burraco

Descrizione del problema

Al termine di ogni round il sistema deve analizzare tutte le combinazioni calate dai giocatori, identificare quelle composte da almeno sette carte e classificarle come Burraco Pulito (privo di matte, o con un "2" in posizione naturale) o Burraco Sporco (contenente Jolly o Pinelle usate come jolly). La sfida risiede nel dover eseguire questa scansione su collezioni dinamiche cresciute nel corso del round, garantendo che la classificazione sia corretta anche nei casi limite, come una scala con un "2" che potrebbe essere sia naturale che matta.

Soluzione proposta

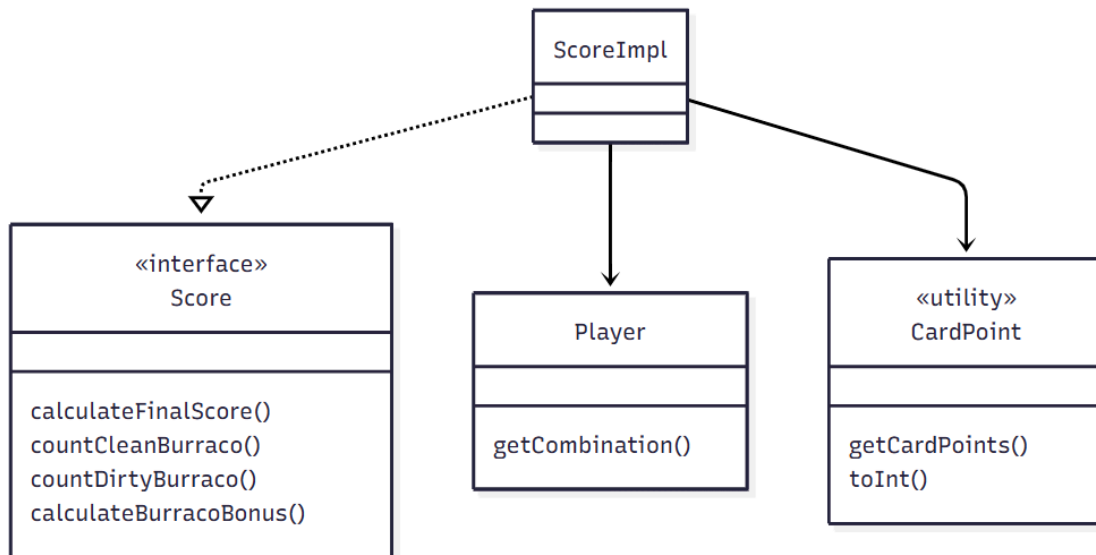
La logica di classificazione è contenuta interamente in `ScoreImpl` tramite il metodo privato `isCleanBurraco` e il suo ausiliario `isTwoInNaturalPosition`, che verifica autonomamente se un "2" occupa la sua posizione naturale nella sequenza. Questo approccio mantiene il calcolo del punteggio autosufficiente e indipendente dalla fase di validazione. Il tipo di Burraco determina il bonus applicato: 200 punti per il Pulito, 100 per lo Sporco.

Analisi della soluzione proposta

Mantenere questa logica autonomamente in `ScoreImpl`, separata da `StraightValidator`, garantisce che il layer di scoring sia indipendente dal layer di validazione. Una modifica alle regole di validazione durante il gioco non si propaga inaspettatamente al calcolo del punteggio di fine round e viceversa, rendendo i due contesti modificabili in isolamento.

Analisi delle alternative scartate

Si era considerato di mantenere un flag di tipo Burraco aggiornato dinamicamente durante il gioco, ogni volta che una carta veniva aggiunta a una combinazione. Questa soluzione è stata scartata perché richiedeva di aggiornare lo stato di classificazione ad ogni mossa, aumentando la complessità e il rischio di disallineamenti in caso di operazioni concorrenti sulla stessa combinazione.



Gestione della pesca multipla dagli scarti

Descrizione del problema

A differenza della pesca dal mazzo, che comporta l'acquisizione di una singola carta, raccogliere la pila degli scarti nel Burraco significa trasferire un numero variabile di carte (potenzialmente l'intera pila accumulata nel corso del round) alla mano del giocatore. La criticità tecnica risiede nel garantire che questo trasferimento avvenga come operazione atomica: tutte le carte devono essere rimosse dalla [DiscardPile](#) e aggiunte alla mano del giocatore in un unico passaggio logico, evitando stati intermedi in cui alcune carte risultino assenti da entrambe le strutture o presenti in entrambe simultaneamente.

Soluzione proposta

L'operazione di raccolta degli scarti è gestita come transazione indivisibile all'interno di [GameModelImpl](#), nel metodo `applyMove`. Quando il giocatore sceglie di raccogliere la pila, [GameModelImpl](#) recupera l'intera collezione tramite `discardPile.getCards()`, la trasferisce alla mano del giocatore con una singola chiamata che itera sulle carte, e svuota atomicamente la pila tramite `discardPile.reset()`. Nessuna operazione intermedia è visibile all'esterno: il `GameState` prodotto dopo l'esecuzione

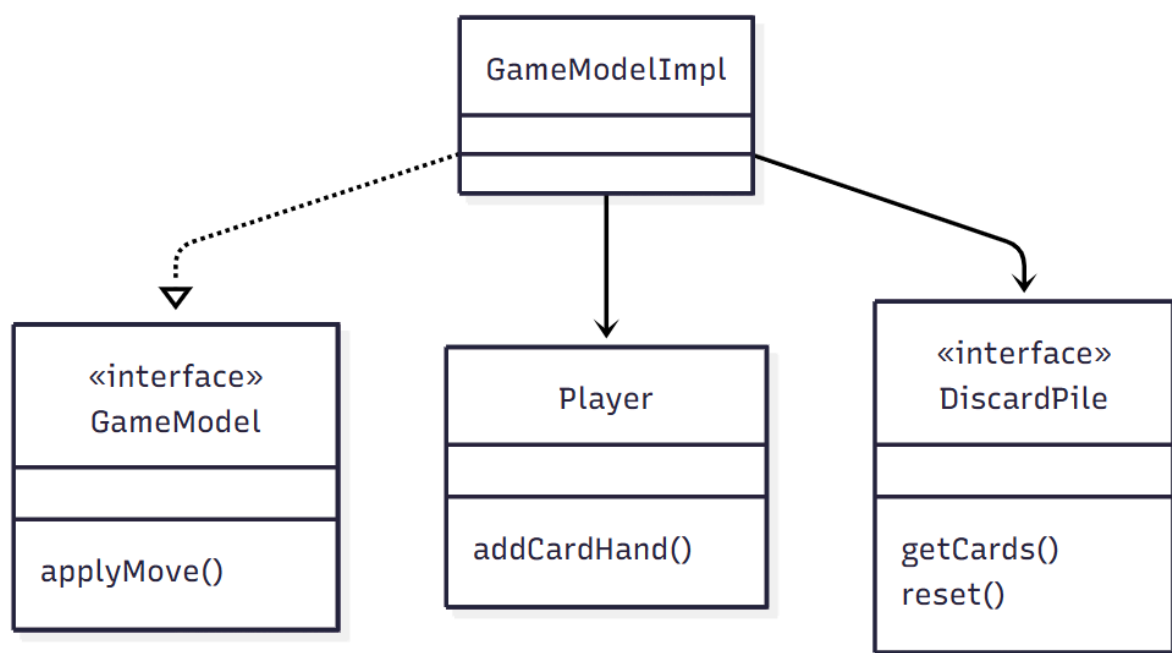
riflette già la situazione aggiornata, garantendo che la View mostri sempre uno stato consistente.

Analisi della soluzione proposta

Trattare la raccolta degli scarti come operazione atomica ha eliminato alla radice il rischio di disallineamento tra Model e View. La scelta di esporre un metodo dedicato su [DiscardPileImpl](#) ha reso impossibile eseguire la raccolta in modo parziale per errore.

Analisi delle alternative scartate

Si era considerato di esporre la lista degli scarti direttamente e lasciare al Controller il compito di iterarla e trasferire le carte una per una. Questa alternativa è stata scartata perché esponeva la struttura interna della [DiscardPile](#) e aumentava il rischio di operazioni parziali in caso di eccezioni durante il trasferimento.



Garanzia della sequenzialità delle azioni di gioco

Descrizione del problema

Nel Burraco le azioni di un turno seguono un protocollo rigido: il giocatore deve obbligatoriamente pescare (dal mazzo o dalla pila degli scarti) prima di poter compiere qualsiasi altra azione, e deve terminare il turno con uno scarto. Consentire un'azione fuori sequenza, come tentare di scartare senza aver pescato, porterebbe il Model in uno stato inconsistente e trasferirebbe alla View la responsabilità di gestire regole di business che non le competono.

Soluzione proposta

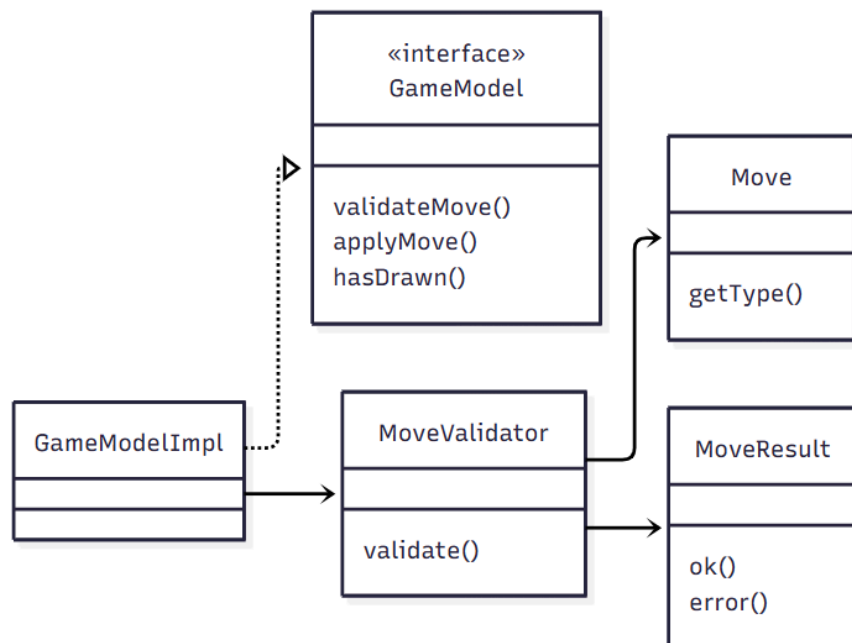
La sequenzialità delle azioni è garantita da **MoveValidator**, che riceve ad ogni validazione il flag `drawnThisTurn` mantenuto da `GameModelImpl`. Prima di autorizzare qualsiasi azione diversa dalla pesca, **MoveValidator** verifica che il giocatore abbia già pescato in questo turno. Se un'azione viene richiesta fuori sequenza, viene restituito un `MoveResult` di errore con status `NOT_DRAWN` o `ALREADY_DRAWN`, che il controller traduce in un **MoveError** e passa alla View per notificare il giocatore, senza alterare lo stato del Model.

Analisi della soluzione proposta

Modellare esplicitamente il ciclo di vita del turno tramite stati ha reso il sistema robusto rispetto a qualsiasi sequenza di input, indipendentemente dalla velocità o dall'ordine con cui l'utente interagisce con la GUI. La View è completamente sollevata dal compito di verificare la legalità della sequenza, limitandosi a mostrare il messaggio di errore ricevuto.

Analisi delle alternative scartate

Si era valutata l'opzione di gestire i controlli di sequenza direttamente nella View, disabilitando i pulsanti non disponibili in base alla fase del turno. Questa soluzione è stata scartata perché avrebbe distribuito logica di business nell'interfaccia grafica, violando la separazione delle responsabilità prevista dall'architettura MVC e rendendo il sistema più fragile in caso di modifiche alle regole.



Giorgia Ceccaroni

Gestione del pozzetto e transizione della mano

Descrizione del problema

Nel Burraco il pozzetto viene acquisito automaticamente nel momento in cui il giocatore esaurisce le carte in mano. Questa transizione può avvenire in due modi distinti con comportamenti differenti: con scarto, dove il giocatore termina il turno scartando l'ultima carta e riceverà il pozzetto all'inizio del turno successivo; oppure al volo, dove il giocatore esaurisce la mano calando una combinazione o attaccando una carta, e può continuare immediatamente il proprio turno giocando dal pozzetto appena ricevuto. Gestire correttamente questi due scenari senza interrompere il flusso del turno nel caso "al volo", o senza dimenticare il cambio di turno nel caso "con scarto", rappresentava una sfida non banale sia lato Model che lato View.

Soluzione proposta

La gestione del pozzetto è stata affidata alla classe [PotManager](#), nel layer controller. Distingue esplicitamente i due scenari tramite un flag booleano `isDiscard` ricevuto al momento della chiamata al metodo `handlePot()`.

Il metodo verifica innanzitutto se il giocatore corrente ha diritto al pozzetto tramite il flag `isInPot()` esposto da `Player`. Se la condizione è soddisfatta, notifica la view tramite [BurracoView](#) con le chiamate `notifyPotTaken` e `markPotTaken`, aggiornando l'indicatore visivo del pozzetto. Nel caso "al volo" (`isDiscard = false`), viene inoltre invocato `refreshHandPanel` per aggiornare immediatamente la mano mostrata a schermo con le nuove carte del pozzetto; nel caso "con scarto" questo aggiornamento non avviene, poiché la mano verrà mostrata al turno successivo.

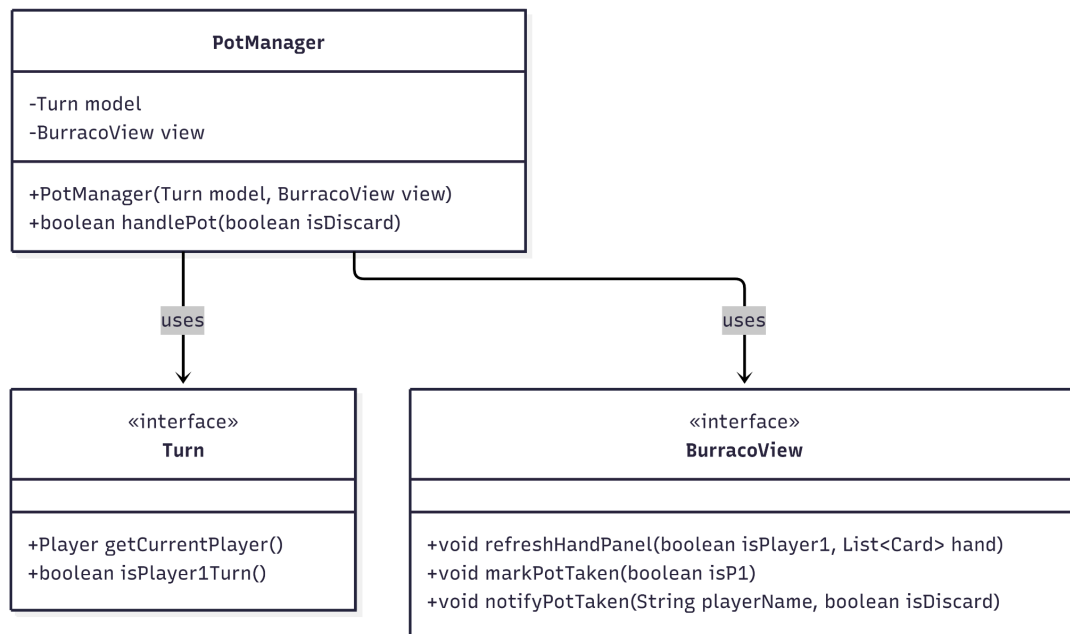


Figura: PotManager, Turn e BurracoView

Analisi della soluzione proposta

Isolare questa logica nel **PotManager** ha evitato di appesantire il **GameLoopController** con condizioni ramificate difficili da seguire. La distinzione esplicita tra i due casi tramite un flag, ha inoltre reso più semplice gestire correttamente l'aggiornamento della View.

L'uso di **Turn** come dipendenza iniettata, invece di **GameModel**, ha mantenuto il coupling al minimo necessario: **PotManager** infatti ha bisogno solo di sapere chi sta giocando e se ha il pozzetto, non dell'intera logica di gioco.

Analisi delle alternative scartate

Si era inizialmente considerato di gestire i casi direttamente nel **GameLoopController** tramite condizioni inline. Questa strada è stata abbandonata perché rendeva il loop principale difficile da leggere e aumentava il rischio di trattare i due scenari in modo incoerente in punti diversi del codice.

Separare la responsabilità in **PotManager** ha invece reso la logica testabile in isolamento, come dimostrato dal **PotManagerTest** che copre i tre scenari principali: pozzetto al volo, pozzetto con scarto, e giocatore senza diritto al pozzetto.

Gestione della chiusura

Descrizione del problema

La chiusura di un round nel Burraco richiede il verificarsi simultaneo di più condizioni: il giocatore deve aver già acquisito il proprio pozzetto, deve aver

realizzato almeno un Burraco e deve necessariamente rimanere con una sola carta in mano da scartare, in modo che una volta effettuato lo scarto la mano risulti completamente vuota. La complessità aggiuntiva risiede nel fatto che queste condizioni devono essere verificate in tre contesti distinti del turno: dopo l'attacco di una carta a una combinazione esistente, dopo il calo di una nuova combinazione e dopo lo scarto. Senza una gestione centralizzata, la logica decisionale avrebbe dovuto essere replicata in ciascuno di questi tre punti, con il concreto rischio di introdurre inconsistenze tra i controlli e di terminare prematuramente un round in modo scorretto. Un ulteriore livello di complessità è rappresentato dalla necessità di riconoscere e denominare tutti i possibili stati intermedi: un giocatore può trovarsi senza carte ma senza pozzetto, oppure con il pozzetto ma senza burraco, e ciascuno di questi stati richiede una risposta diversa da parte del sistema.

Soluzione proposta

Il problema è stato decomposto in due componenti distinti: l'enum [ClosureState](#) e la classe [ClosureValidator](#).

[ClosureState](#) enumera esaustivamente tutti gli stati possibili riguardo alla chiusura: OK (si può giocare normalmente), CAN_CLOSE (pozzetto preso e almeno un burraco: il giocatore può chiudere), CANNOT_CLOSE_NO_BURRACO (mano vuota dopo lo scarto ma senza burraco), ZERO_CARDS_NO_POT (zero carte in mano senza aver preso il pozzetto), ZERO_CARDS_NO_BURRACO (pozzetto preso, mano vuota, ma nessun burraco formato), ROUND_WON (chiusura completata, round vinto). Avere un vocabolario esplicito di stati ha reso il codice leggibile sia per chi scrive che per chi lo mantiene.

La verifica delle condizioni di chiusura è stata centralizzata nella classe [ClosureValidator](#). La valutazione è delegata sistematicamente a [ClosureValidator](#) da [GameModelImpl](#), che la invoca dopo ogni azione rilevante (calo di una combinazione, un attacco, e dopo uno scarto). In questo modo, qualsiasi modifica alle regole di chiusura richiede di intervenire esclusivamente su [ClosureValidator](#), lasciando inalterato il resto del sistema.

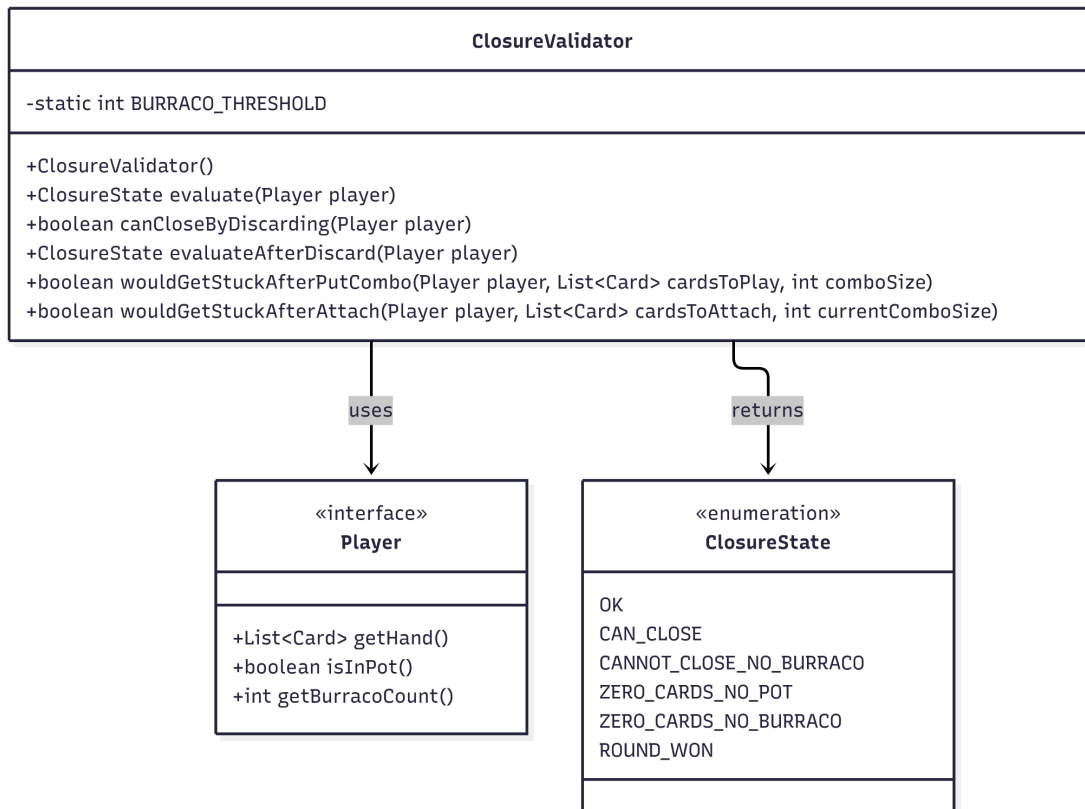


Figura: ClosureValidator, ClosureState e Player

Analisi della soluzione proposta

Centralizzare i controlli in **ClosureValidator** ha eliminato il rischio di duplicazione della logica decisionale. Ogni modifica alle regole di chiusura, come l'aggiunta di una nuova condizione, richiede di intervenire esclusivamente su **ClosureValidator**, lasciando inalterato il resto del sistema. L'enum **ClosureState** ha reso espliciti gli stati intermedi che altrimenti sarebbero rimasti impliciti in condizioni sparse, rendendo ogni scenario testabile con un test dedicato direttamente sul validatore senza dover simulare un turno completo.

Analisi delle alternative scartate

Si era considerata l'ipotesi di distribuire i controlli nei tre punti del turno in cui la chiusura può verificarsi, inserendo le condizioni inline nel codice del game loop. Questa strada è stata abbandonata perché avrebbe inevitabilmente portato a duplicazioni: una modifica alle regole di chiusura avrebbe richiesto di aggiornare il codice in almeno tre punti distinti, aumentando il rischio di dimenticarne uno e introducendo comportamenti incoerenti tra i diversi contesti.

Rilevamento preventivo della situazione di "stuck"

Descrizione del problema

Durante lo sviluppo si è presentato un caso limite critico: un giocatore potrebbe posare carte o effettuare un attacco rimanendo senza carte in mano, ma senza aver ancora realizzato un Burraco e avendo già ottenuto il pozzetto. In questo scenario il giocatore si trova in una situazione di stallo da cui non può uscire: non può chiudere il round, non ha carte da scartare e non ha un pozzetto a disposizione. Rilevare questo problema dopo che la mossa è già stata applicata al model avrebbe richiesto un meccanismo di rollback complesso e invasivo rischiando di compromettere lo stato del gioco.

Soluzione proposta

Il rilevamento preventivo è stato integrato direttamente in [ClosureValidator](#) tramite i metodi `wouldGetStuckAfterPutCombo` e `wouldGetStuckAfterAttach`. Entrambi ricevono il giocatore, le carte che intende giocare e la dimensione della combinazione coinvolta, e restituiscono true se la mossa lascerebbe il giocatore in uno stato irrecuperabile. [MoveValidator](#) invoca questi metodi nella fase di pre-validazione, prima che la mossa venga applicata: se la condizione di stuck viene rilevata, il risultato è un [MoveResult](#) con stato `WOULD_GET_STUCK`, che il controller traduce in un [MoveError](#) e mostra alla view, lasciando il model invariato.

La logica di stuck è calibrata tenendo conto del threshold del burraco (7 carte): se il calo o l'attacco porterebbe la combinazione ad almeno 7 carte, formando un burraco nel medesimo atto, la condizione non scatta, perché il giocatore avrebbe soddisfatto il requisito per chiudere.

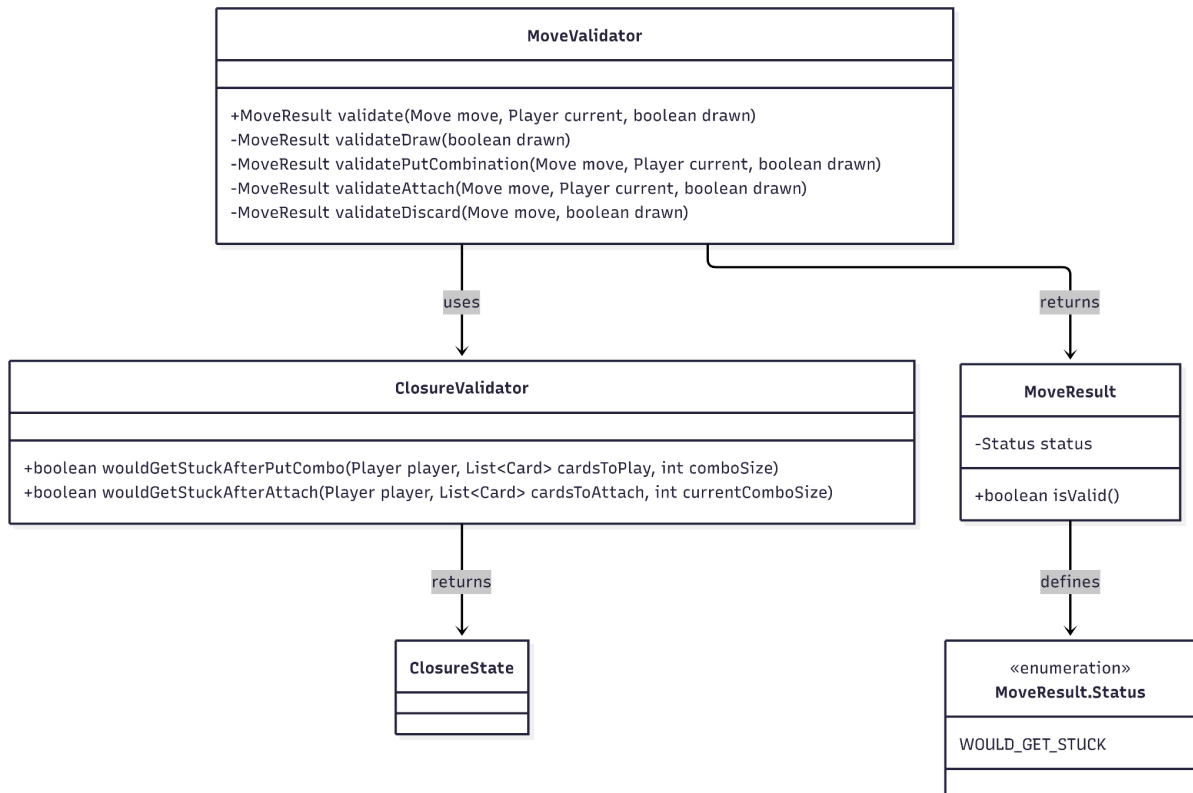


Figura: ClosureValidator, MoveValidator e MoveResult

Analisi della soluzione proposta

Anticipare il controllo alla fase di pre-validazione ha garantito che il Model non raggiunga mai uno stato inconsistente. Questo approccio è coerente con il principio generale adottato nel sistema di centralizzare i controlli di validità prima dell'applicazione delle mosse, mantenendo il Model sempre in uno stato coerente con le regole del gioco, indipendentemente dalla sequenza di input dell'utente.

Analisi delle alternative scartate

Si era considerato di rilevare la situazione di stuck dopo l'applicazione della mossa e di gestirla tramite un meccanismo di rollback. Questa alternativa è stata scartata perché introdurre un rollback avrebbe richiesto di rendere reversibili tutte le operazioni sul Model, avrebbe richiesto di replicare o annullare ogni effetto collaterale (rimozione carte dalla mano, aggiornamento delle combinazioni sul tavolo) aumentando significativamente la complessità e il rischio di stati inconsistenti durante il ripristino.

Gestione dell'alternanza dei turni

Descrizione del problema

Nel gioco del Burraco i due giocatori si alternano in modo rigoroso: ogni azione, dalla pesca allo scarto, è iscritta nel contesto di un turno ben preciso, e il sistema deve essere in grado di sapere in ogni istante a chi spetta agire. La sfida non risiede soltanto nell'alternare i giocatori, ma nel garantire che il modello esponga sempre un'unica fonte di verità sullo stato corrente del turno, accessibile da tutti i componenti che ne hanno bisogno ([GameModel](#), [PotManager](#), [ClosureValidator](#)) senza che nessuno di essi debba mantenere una propria copia locale dello stato.

Soluzione proposta

La gestione dell'alternanza è stata incapsulata nell'interfaccia [Turn](#) e nella sua implementazione [TurnImpl](#). L'interfaccia espone un contratto minimale che racchiude tutte le informazioni necessarie per il corretto svolgersi del turno di gioco: `nextTurn()` per avanzare al giocatore successivo, `getCurrentPlayer()` per conoscere il giocatore attivo, `isPlayer1Turn()` per distinguere i due giocatori, e `resetForNewRound()` per ripristinare lo stato all'inizio di un nuovo round.

[TurnImpl](#) realizza questo contratto servendosi prevalentemente di un unico flag booleano `isPlayer1Turn`, invertito da `nextTurn()`, eliminando qualsiasi possibilità di disallineamento tra lo stato dichiarato e quello effettivo.

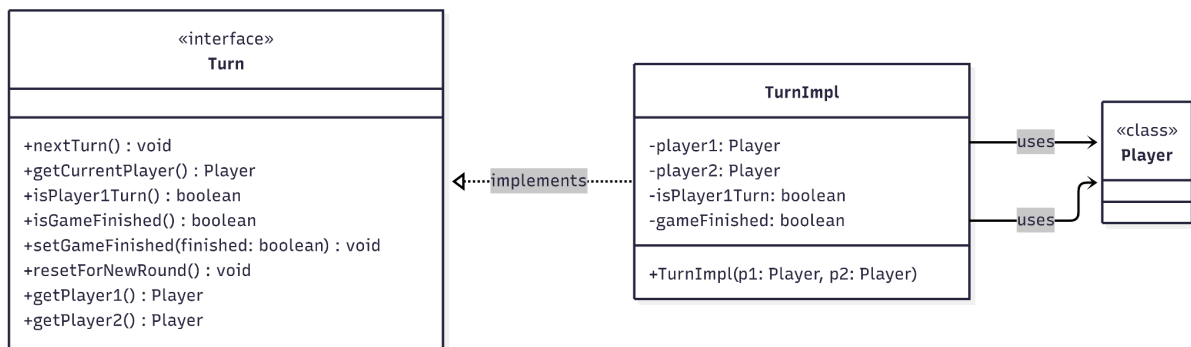


Figura: Interfaccia Turn e implementazione TurnImpl

Analisi della soluzione proposta

Separare l'interfaccia dall'implementazione ha permesso di sostituire [TurnImpl](#) con varianti diverse senza modificare nessun componente che usa [Turn](#). Il flag booleano come unica sorgente di verità rende impossibile che i due giocatori risultino entrambi attivi o entrambi inattivi contemporaneamente.

Il metodo `resetForNewRound()` garantisce che, a ogni nuova manche, il turno parta sempre da Player 1 con il flag `gameFinished` azzerato, senza che `GameModel` o `RoundController` debbano ricordarsi di ripristinare manualmente questi valori.

Analisi delle alternative scartate

Si era valutata l'ipotesi di mantenere lo stato del turno direttamente in `GameModelImpl`, usando un campo booleano locale. Questa soluzione è stata scartata perché avrebbe costretto tutti i componenti che necessitano dello stato di turno a dipendere dall'intera interfaccia `GameModel`, creando accoppiamenti non necessari e rendendo più difficile testare la logica di alternanza in isolamento.

3. Sviluppo

3.1 Testing automatizzato

Il progetto include una suite di classi di test realizzate con il framework JUnit 5, distribuite nei package `test.model` e `test.controller`, specchiando la struttura del codice sorgente.

L'esecuzione dei test è automatizzata tramite Gradle: è sufficiente lanciare il task `test` perché l'intera suite venga eseguita senza alcun intervento manuale, producendo un report dettagliato dei risultati. Questo approccio garantisce una verifica sistematica e ripetibile della correttezza del software ad ogni modifica del codice.

Le classi di test sono organizzate in due macro-aree: il package `model` raccoglie i test sulle entità di dominio del gioco, quali carte, mazzo, pila degli scarti, mosse, giocatori, regole di validazione e punteggio; il package `controller` contiene invece i test relativi alla logica applicativa, tra cui la gestione della distribuzione, del piatto, dei turni di gioco e del punteggio.

Per ciascuna classe di implementazione sono stati sviluppati metodi di test mirati a coprire le diverse casistiche di comportamento, verificando sia i casi nominali sia le condizioni al contorno, così da assicurare che ogni componente rispetti i requisiti del gioco del Burraco.

Un elemento centrale della strategia di testing è l'utilizzo di Mockito, che ha permesso di isolare ogni unità in fase di verifica attraverso la creazione di oggetti mock. Questa tecnica si è rivelata particolarmente importante per i controller e i gestori della logica di gioco, consentendo di simulare il comportamento delle dipendenze, come il mazzo, la mano del giocatore o la pila degli scarti, senza dover istanziare l'intero sistema, rendendo i test più veloci, stabili e focalizzati.

Le componenti sottoposte a test automatico sono:

Model:

CardTest: Verifica il comportamento della classe `CardImpl`, che rappresenta una singola carta da gioco. I test accertano che il costruttore inizializzi correttamente seme e valore della carta, che la gestione dello stato jolly funzioni come atteso e che la rappresentazione testuale sia corretta sia per le carte normali che per il jolly. Viene inoltre verificato che il valore numerico restituito sia coerente con il valore della carta, incluso il caso speciale del jolly, che restituisce 0.

DeckTest: Verifica il comportamento della classe `DeckImpl`, che rappresenta il mazzo di carte. I test controllano che il mazzo venga inizializzato con 108 carte nella composizione corretta (2 copie per ogni carta, 4 jolly), che la pesca riduca

correttamente la dimensione e che il reset ripristini il mazzo completo, anche quando era vuoto.

DiscardPileTest: Verifica il comportamento della pila degli scarti. I test controllano che lo stato iniziale sia vuoto, che l'aggiunta di carte funzioni correttamente, che il reset svuoti la pila e che la pesca da una pila vuota restituisca null.

MoveTest: Verifica la classe Move e MoveResult. I test accertano che una mossa venga costruita correttamente con tipo e carte selezionate, che le liste restituite siano immutabili e che i vari stati di MoveResult abbiano la validità attesa.

PlayerTest: Verifica il comportamento della classe PlayerImpl. I test coprono la gestione della mano, l'interazione con il tavolo, la persistenza del punteggio tra i round, il reset per un nuovo round e la correttezza della copia difensiva nelle combinazioni.

ClosureValidatorTest: Verifica che il validatore di chiusura riconosca correttamente tutti gli stati possibili di fine turno: restituisce OK quando il giocatore ha ancora carte in mano, ZERO_CARDS_NO_POT quando ha esaurito le carte senza aver pescato il piatto, ZERO_CARDS_NO_BURRACO quando ha pescato il piatto ma non ha ancora completato un burraco, e CAN_CLOSE quando tutte le condizioni per chiudere il round sono soddisfatte.

CombinationValidatorTest: Verifica la validazione delle combinazioni di carte nelle sue casistiche principali. I test controllano che una combinazione troppo corta venga rifiutata, che un tris di carte dello stesso valore e semi diversi venga accettato, che le scale dello stesso seme in sequenza siano valide e che l'uso di jolly e del 2 come matta venga gestito correttamente rispettando il limite di una sola matta per combinazione.

SetHandlerTest: Verifica la logica di validazione dei tris. I test controllano che un tris di carte con lo stesso valore e semi diversi sia valido, che valori diversi lo rendano invalido e che la presenza di due jolly nella stessa combinazione venga rifiutata.

StraightUtilsTest: Verifica le regole di validazione delle sequenze. I test controllano che una scala dello stesso seme sia valida, che un jolly possa sostituire una carta mancante e che una sequenza con semi misti venga rifiutata.

CardPointTest: Verifica l'assegnazione dei punteggi alle carte. I test controllano che jolly, due e asso valgano rispettivamente 30, 20 e 15 punti, che le carte alte valgano 10

e quelle basse 5, e che la mappatura da carta a intero sia corretta, inclusa l'eccezione per il jolly.

ScoreTest: Verifica il calcolo del punteggio finale. I test coprono il valore delle carte in mano e sul tavolo, il riconoscimento e il conteggio di burracci puliti e sporchi, l'applicazione della penalità per mancata pesca dal piatto e il calcolo complessivo del punteggio di fine round.

TurnTest: Verifica la gestione dei turni di gioco. I test controllano l'alternanza corretta tra i due giocatori, le condizioni necessarie per chiudere il round (burraco completato e piatto pescato) e il reset dello stato per un nuovo round.

GameModelTest: Verifica il funzionamento del modello di gioco nella sua interezza, testando l'integrazione tra i componenti. I test coprono lo stato iniziale della partita, la transizione di turno, la pesca dal mazzo, il divieto di pescare due volte nello stesso turno, lo scarto di una carta e la corretta delega della validazione delle mosse a MoveValidator, incluso il rifiuto di una combinazione vuota.

Controller

DistributionManagerImplTest: Verifica che la distribuzione iniziale delle carte avvenga correttamente. Tramite oggetti mock, il test accerta che a ciascun giocatore vengano distribuite 11 carte in mano e 11 nel piatto, che una carta venga aggiunta alla pila degli scarti e che il mazzo venga pescato esattamente 45 volte in totale.

InitialDistributionControllerTest: Verifica che il controller di distribuzione iniziale deleghi correttamente la propria logica al DistributionManagerImpl. Il test accerta che il metodo distribute invochi effettivamente distributeInitialCards sul manager con gli stessi parametri ricevuti.

PotManagerTest: Verifica la gestione della pesca dal piatto. I test coprono tre scenari: il giocatore pesca il piatto direttamente, il giocatore lo pesca dopo uno scarto, e il giocatore non ha diritto al piatto. In ciascun caso si verifica che la view riceva le notifiche corrette e che lo stato venga aggiornato di conseguenza.

ResetManagerTest: Verifica che il reset di fine round coinvolga tutti i componenti necessari. Il test accerta che vengano invocati resetForNewRound su entrambi i giocatori e reset sia sul mazzo che sulla pila degli scarti.

RoundControllerTest: Verifica l'orchestrazione completa di un nuovo round. Il test accerta che il controller esegua nell'ordine corretto il reset dello stato, l'aggiornamento della view, la distribuzione delle carte e il ridisegno della tabella di gioco.

ScoreControllerTest: Verifica il comportamento del controller del punteggio a fine round. I test accertano che il punteggio venga calcolato per entrambi i giocatori e aggiunto al totale della partita, e che la registrazione di un'azione per il nuovo round venga correttamente inoltrata al RoundEndHandler.

3.2 Note di sviluppo

Le sottosezioni seguenti elencano, per ciascun membro, le funzionalità avanzate del linguaggio Java utilizzate.

Beatrice Esposto

Stream e Lambda expressions

Utilizzate pervasivamente nella validazione delle scale.

Permalink:

<https://github.com/chiarama05/OOP25-Burraco/blob/5ce1bc302eab5c5fa34bb2a66ef67e42e26766d5/src/main/java/it/unibo/burraco/model/rules/StraightValidator.java#L109>

AtomicInteger

Utilizzato in CardImpl per garantire l'assegnazione thread-safe di un identificatore univoco a ogni carta.

Permalink:

<https://github.com/chiarama05/OOP25-Burraco/blob/1ff3cd8bb0d367e0da6822db7a1802b60fe8240b/src/main/java/it/unibo/burraco/model/cards/CardImpl.java#L12>

Sofia Mascioli

Stream e Lambda expressions

Utilizzate pervasivamente nella validazione delle combinazioni e nella gestione delle carte attaccabili.

Permalink:

<https://github.com/chiarama05/OOP25-Burraco/blob/d188ae847f3e54e3be9187e436a0603a5ebcf894/src/main/java/it/unibo/burraco/model/rules/CombinationValidator.java#L44>

Permalink:

<https://github.com/chiarama05/OOP25-Burraco/blob/1ff3cd8bb0d367e0da6822db7a1802b60fe8240b/src/main/java/it/unibo/burraco/model/rules/CombinationValidator.java#L44>

Stream Collectors

Utilizzati in CombinationValidator per la raccolta e deduplicazione dei valori delle carte.

Permalink:

<https://github.com/chiararama05/OOP25-Burraco/blob/d188ae847f3e54e3be9187e436a0603a5ebcf894/src/main/java/it/unibo/burraco/model/rules/CombinationValidator.java#L103>

Permalink:

<https://github.com/chiararama05/OOP25-Burraco/blob/d188ae847f3e54e3be9187e436a0603a5ebcf894/src/main/java/it/unibo/burraco/model/rules/CombinationValidator.java#L107>

Chiara Mazzoli

Stream e Lambda expressions

Utilizzate pervasivamente nel calcolo del punteggio finale.

Permalink:

<https://github.com/chiararama05/OOP25-Burraco/blob/5ce1bc302eab5c5fa34bb2a66ef67e42e26766d5/src/main/java/it/unibo/burraco/model/score/ScoreImpl.java#L94>

CompletableFuture

Utilizzato per la sincronizzazione tra il game loop e le azioni dell'utente sulla GUI.

Permalink:

<https://github.com/chiararama05/OOP25-Burraco/blob/1ff3cd8bb0d367e0da6822db7a1802b60fe8240b/src/main/java/it/unibo/burraco/view/table/TableViewImpl.java#L153>

Java Record

ScoreSnapshot è implementato come Java Record per garantire l'immutabilità del DTO tra controller e view.

Permalink:

<https://github.com/chiararama05/OOP25-Burraco/blob/d188ae847f3e54e3be9187e436a0603a5ebcf894/src/main/java/it/unibo/burraco/controller/dto/ScoreSnapshot.java#L26>

Giorgia Ceccaroni

Le classi di mia competenza non fanno uso di funzionalità avanzate del linguaggio Java nel senso indicato (osservando le indicazioni nel template per la relazione di progetto). La complessità del contributo risiede interamente nella logica di dominio: la gestione dell'alternanza dei turni, la centralizzazione dei controlli di chiusura e per il pozzetto, e il rilevamento preventivo della situazione di stallo sono problemi affrontati attraverso un design attento piuttosto che attraverso l'uso di API particolari del linguaggio.

4. Commenti finali

4.1 Autovalutazione e lavori futuri

L'obiettivo principale di questo progetto era la realizzazione di una replica fedele del gioco del Burraco, un compito che si è rivelato estremamente stimolante a causa della densità delle regole e della complessità nella gestione dei casi particolari. L'applicazione finale soddisfa tutti i requisiti funzionali previsti, tra cui la gestione dei pozzetti, la validazione delle combinazioni e il calcolo del punteggio cumulativo tra i round. L'adozione del pattern architetturale MVC si è rivelata fondamentale sia per permettere al gruppo di lavorare in parallelo in modo efficace, sia per garantire una netta separazione tra la logica di validazione del Model e la reattività della View, rendendo il sistema più manutenibile ed estensibile.

Beatrice Esposto

Il mio contributo si è concentrato sulla gestione del mazzo e sulla logica di validazione delle scale. Per l'inizializzazione del mazzo ho garantito che le 108 carte venissero istanziate correttamente come oggetti distinti, risolvendo il problema dell'unicità tramite l'introduzione di un identificatore numerico univoco generato in modo thread-safe ad ogni istanziazione, assicurandomi che due carte con lo stesso seme e valore siano sempre distinguibili dal sistema. Per la validazione delle scale ho implementato la logica di ordinamento delle scale e la gestione dell'ambiguità semantica del "2" e del Jolly all'interno delle scale, eseguendo la verifica in due passaggi distinti: prima trattando il "2" come carta naturale, poi forzandolo come wildcard, accettando la combinazione se almeno uno dei due percorsi produce una sequenza valida. Mi sono inoltre occupata della validazione degli attacchi a combinazioni esistenti, garantendo che le carte aggiungibili a una scala rispettassero le stesse regole applicate in fase di creazione, senza duplicare la logica già presente. Sono soddisfatta delle soluzioni adottate: separare queste responsabilità in componenti dedicati ha evitato duplicazioni, reso il codice testabile in isolamento e ridotto il rischio di inconsistenze nella gestione dell'identità delle carte.

Giorgia Ceccaroni

Il mio contributo ha riguardato la gestione dei turni, la logica di chiusura del round, il rilevamento preventivo della situazione di stallo e la gestione dei pozzetti. Mi sono occupata dell'alternanza corretta tra i giocatori tramite l'interfaccia Turn e la sua implementazione TurnImpl, dove un unico flag booleano costituisce la sorgente di verità sull'attore corrente, eliminando ogni possibilità di disallineamento tra i componenti che ne dipendono.

Per la chiusura, ho scelto di separare il vocabolario degli stati (ClosureState) dalla logica di valutazione (ClosureValidator), centralizzando tutti i controlli in un unico punto interrogabile nei tre contesti del turno in cui la chiusura può verificarsi: dopo un calo, dopo un attach e dopo uno scarto. Questa scelta ha reso ogni scenario limite testabile in isolamento, come dimostrato dal ClosureValidatorTest.

Il rilevamento preventivo della situazione di stuck è stato integrato nella fase di pre-validazione tramite i metodi wouldGetStuckAfterPutCombo e wouldGetStuckAfterAttach, garantendo che il model non raggiunga mai uno stato inconsistente senza dover introdurre alcun meccanismo di rollback.

Per i pozzetti, ho distinto esplicitamente la presa al volo dalla presa con scarto affidando questa logica al PotManager, che riceve un flag booleano e sa esattamente quale aggiornamento notificare alla view, mantenendo il GameLoopController snello. Sono soddisfatta di queste scelte architetturali, che hanno reso ogni componente testabile in modo indipendente e hanno limitato le dipendenze al minimo strettamente necessario.

Sofia Mascioli

Il mio contributo ha riguardato l'inizializzazione delle carte tramite gli enum Seed e CardValue, la verifica delle combinazioni valide e la visualizzazione delle combinazioni sul tavolo. L'introduzione degli enum ha eliminato tutti i confronti su stringhe che erano presenti nel codice, rendendolo più robusto e leggibile. Per la validazione, ho separato la logica dei set da quella delle scale delegando a componenti distinti, SetHandler e StraightValidator, permettendo di testare le due tipologie in modo completamente indipendente. Per la visualizzazione, ho sviluppato CombinationDisplaySorter e gestito il ridisegno completo della BoardView ad ogni aggiornamento di stato, garantendo che le combinazioni mostrate siano sempre allineate con il Model. Sono soddisfatta di queste scelte, che rendono il sistema flessibile e coerente dalla validazione fino alla visualizzazione.

Chiara Mazzoli

Il mio contributo ha riguardato la distribuzione iniziale delle carte, la gestione della pesca dal mazzo e dagli scarti, la logica di scarto e il sistema di punteggio. Mi sono occupata della distribuzione iniziale tramite DistributionManagerImpl, garantendo che ogni giocatore ricevesse le carte corrette in mano e nel pozzetto. Per la pesca dagli scarti ho implementato un trasferimento atomico dell'intera pila, eliminando il rischio di stati inconsistenti tra Model e View. Per il punteggio, ho centralizzato la logica di valorizzazione delle carte in CardPoint, mantenendo il modello snello e rendendo semplice modificare i valori senza toccare altre componenti. Sono soddisfatta delle scelte adottate, in particolare dell'approccio atomico alla raccolta degli scarti e della separazione tra logica di punteggio e rappresentazione della carta.

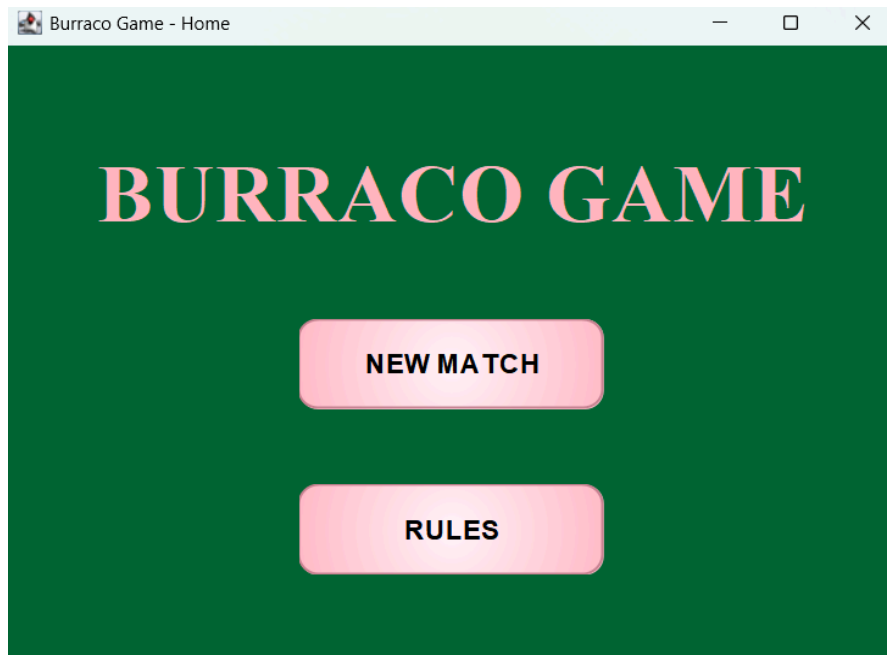
4.2 Sviluppi futuri

Il sistema realizzato costituisce una base solida e ben strutturata, su cui è possibile innestare diverse estensioni senza stravolgere l'architettura esistente. Tra i possibili sviluppi futuri abbiamo identificato:

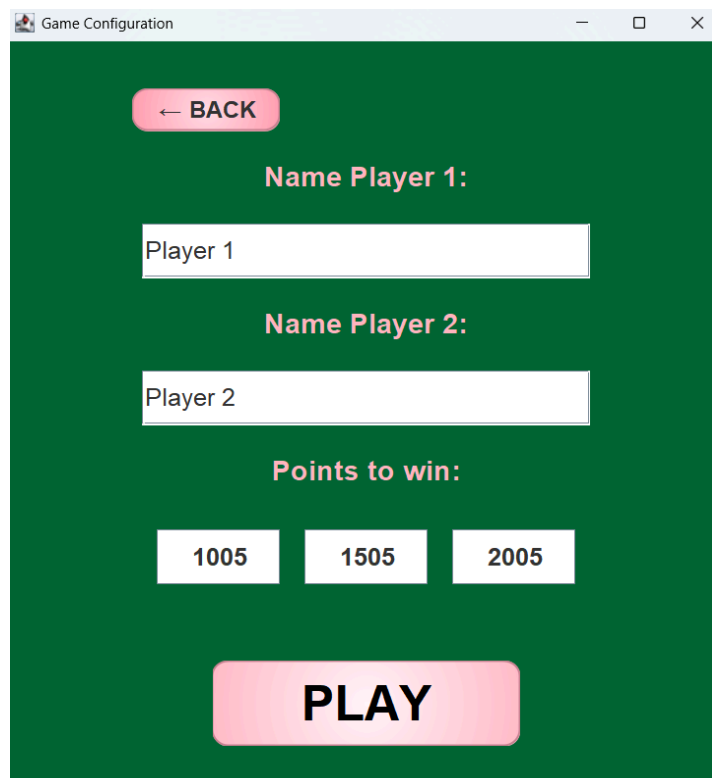
- **Avversario artificiale:** introduzione di un giocatore controllato dall'AI con livelli di difficoltà progressivi, dalla strategia casuale fino a un agente in grado di valutare le combinazioni ottimali e tenere traccia delle carte già giocate.
- **Modalità a quattro giocatori:** supporto per partite in coppia, la modalità più diffusa del Burraco reale, che richiederebbe principalmente l'estensione della gestione dei turni e del punteggio condiviso tra compagni.
- **Salvataggio della partita:** serializzazione dello stato del Model per permettere di salvare e ripristinare una partita interrotta.
- **Modalità torneo:** oltre al salvataggio del punteggio, già presente e funzionante, per implementare questa feature futura, si renderebbe necessaria la sola implementazione di un database per gestire uno storico dettagliato delle partite precedenti, in modo da creare un sistema di tornei (simile ragionamento si potrebbe utilizzare per creare una classifica globale dei giocatori).
- **Modalità in rete:** Grazie al disaccoppiamento tra Model e View, sarebbe possibile sostituire TableView con un'implementazione che comunica via socket o WebSocket con un secondo client, senza modificare il Controller né il Model.
- **Ottimizzazione della logica di scarto:** Attualmente il gioco prevede l'interazione umana per ogni mossa; si potrebbe implementare un sistema di suggerimenti basato sulla probabilità di pesca di determinate carte.

5. Guida utente

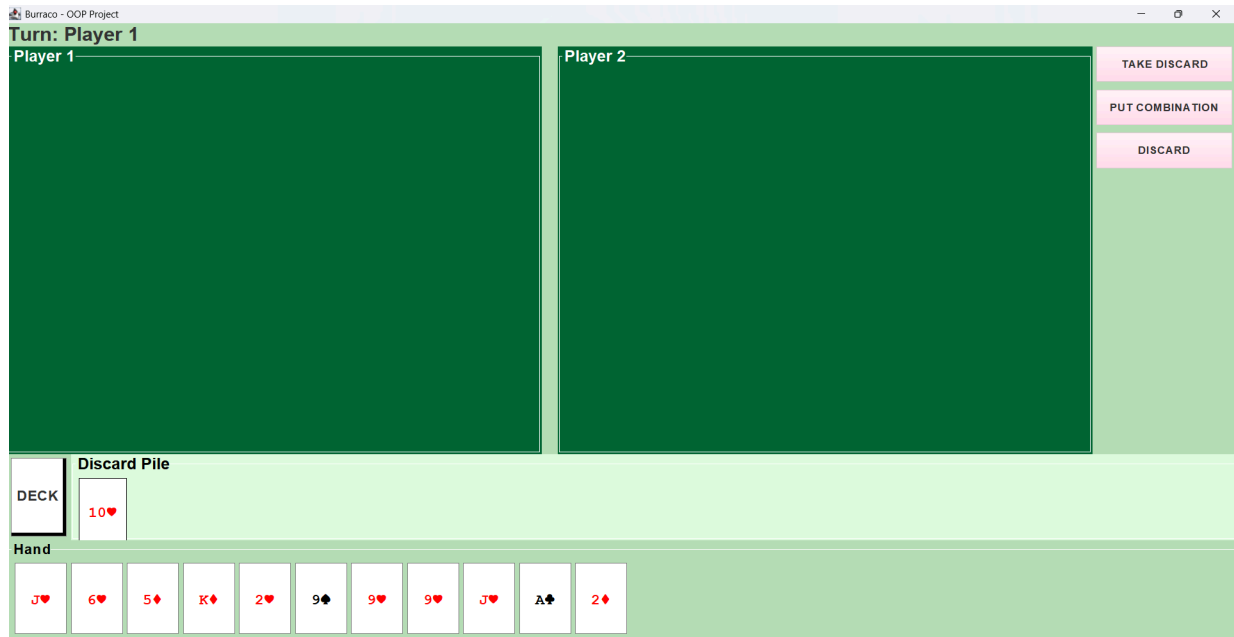
All'avvio del gioco ci si troverà di fronte a questa schermata.



- **New Match:** porterà alla schermata per settare i nomi e il punteggio della partita
- **Rules:** porterà ad una schermata contenente le regole del gioco

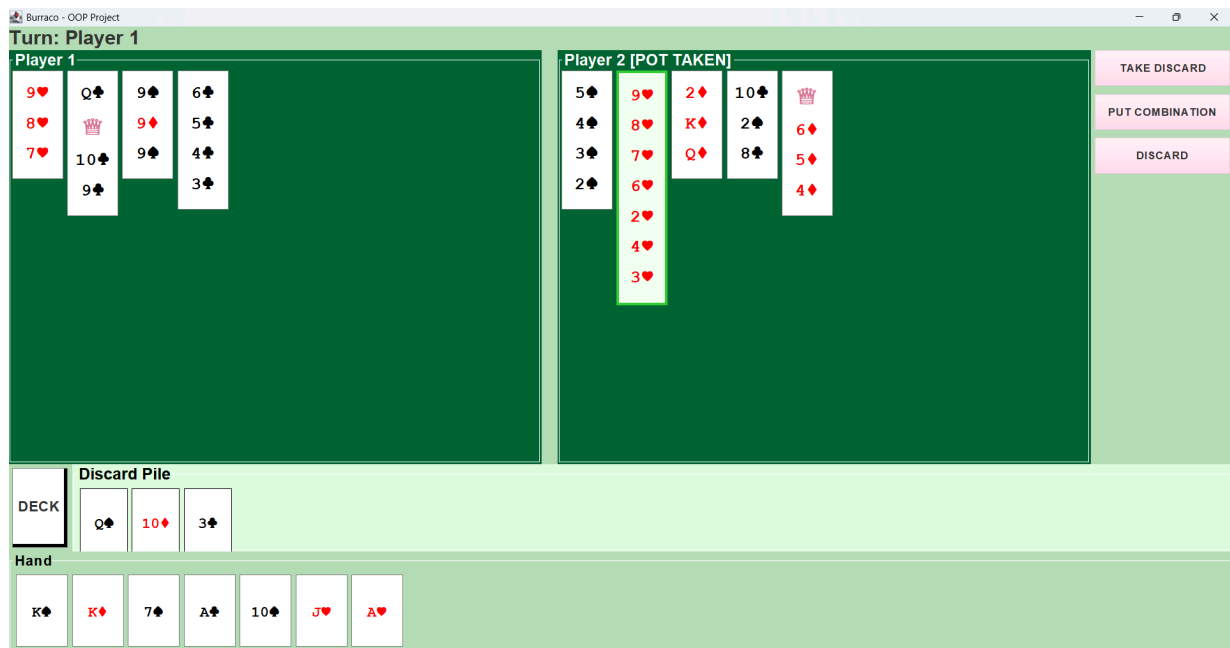


- **Back:** torna alla schermata iniziale
- **Name Player1/Player2:** sono campi testuali dove si possono inserire i nomi dei giocatori.
- **Points to win:** scelta del punteggio, ovvero l'obiettivo in termini di punti che un giocatore deve raggiungere per vincere la partita
- **Play:** dà il via alla/e partita/e



- **Deck:** pulsante per pescare una carta dal mazzo, è obbligatorio pescare prima di qualsiasi altra azione.
- **Take discard:** pulsante per pescare tutte le carte presenti nella discard pile
- **Put combination:** pulsante che permette ad un giocatore di calare sul tavolo la combinazione di carte dopo averle selezionate dalla propria Hand, la combinazione deve essere una scala o un tris di minimo 3 carte.
- **Discard:** pulsante per scartare una ed una sola carta precedentemente selezionata dalla propria Hand; questa azione termina il turno.

Nota: quando il turno cambia, appare un messaggio di dialogo che chiede al giocatore attivo di aver preso visione del messaggio (premendo OK) prima di mostrare la mano dell'altro giocatore, in modo che così l'avversario non possa vedere le carte del player subentrante.



- **Pot taken:** didascalia che compare nel momento in cui il giocatore prende il pozzetto
- **Combinazioni:** sono rappresentate dalle scale e dai tris già presenti sul tavolo di gioco, sono state implementate come bottoni, e permettono al giocatore di attaccare una carta precedentemente selezionata dalla Hand
- **Combinazione illuminata di colore verde:** indica il Burraco

Burraco - Final Standings

SCOREBOARD

PLAYER 1		PLAYER 2	
Cards on Table	375	Cards on Table	235
Clean Burraco	200	Clean Burraco	200
Dirty Burraco	0	Dirty Burraco	100
Closure Bonus	100	Closure Bonus	0
Pot Penalty	0	Pot Penalty	0
Cards in Hand	-0	Cards in Hand	-10
ROUND TOTAL		ROUND TOTAL	
MATCH TOTAL		MATCH TOTAL	
675		525	
675		525	

NEXT ROUND (Target: 1005 pts)

- **Next Round:** bottone per iniziare un nuovo round finchè non si raggiunge il punteggio target selezionato all'inizio



The screenshot shows a window titled "Burraco - Final Standings" with a green background. At the top, the word "SCOREBOARD" is written in large, bold, pink letters. Below this, the scores for two players are displayed. Player 1 is on the left, marked with a trophy icon, and Player 2 is on the right. Each player's score is broken down into several categories: Cards on Table, Clean Burraco, Dirty Burraco, Closure Bonus, Pot Penalty, and Cards in Hand. At the bottom of each column, the "ROUND TOTAL" and "MATCH TOTAL" are shown. A large pink button at the bottom of the window reads "CHAMPION: PLAYER 1 (FINISH GAME)".

PLAYER 1		PLAYER 2	
Cards on Table	245	Cards on Table	95
Clean Burraco	200	Clean Burraco	0
Dirty Burraco	100	Dirty Burraco	0
Closure Bonus	100	Closure Bonus	0
Pot Penalty	0	Pot Penalty	-100
Cards in Hand	-0	Cards in Hand	-40
ROUND TOTAL	645	ROUND TOTAL	-45
MATCH TOTAL	1320	MATCH TOTAL	480

CHAMPION: PLAYER 1 (FINISH GAME)

SCHEMATA FINALE:

- **Champion:** bottone che indica il nome del vincitore e chiude l'applicazione